What Does Formal Structure in Counterexample Feedback Buy?

Neel Mistry
neel@mistryneel.com

Sreedath Panat
sreedath@vizuara.com

Abstract

Counterexample-guided repair asks a synthesizer to fix a program given a description of how it failed. When that synthesizer is a large language model, the description can range from a raw execution trace to a formal certificate carrying a violated predicate, a signed robustness value, a minimized falsifying input, and the program location responsible. We hold the falsifier, the program, the counterexample corpus, the token budget and the evaluation suite fixed, vary only the rendering of the failure across five conditions, and run 300 one-step repair attempts against an interpretable behaviour-tree policy for an unprotected left turn in CARLA. The conditions differ sharply in what kind of repair is attempted: structural edits account for $59/60$ attempts under a raw trace against $13/60$ under a natural-language summary, a difference of 76.7 points with 95% interval $[+61.7, +90.0]$. Because the raw trace also differs from the other conditions in length and in information content, we report this as a difference between feedback conditions, not as an effect of formal structure. The central finding concerns the success criterion itself. Satisfying the safety contracts on the failing scenario is trivially achievable by refusing to act: of the 18 attempts that satisfied the contracts, 13 never entered the junction's conflict zone; of the other 5, 3 completed the manoeuvre, and 1 of those also left the regression suite intact. That one program satisfies the contracts and completes the turn on all twenty counterexamples, so the language can express a repair that passes this corpus. Why it was found only once, whether because of search difficulty, the baseline's structure, or a criterion that rewards inaction, remains open. Excluding refusals, we did not detect a reliable improvement in repair success from any condition, and under stated assumptions only large improvements would reliably have been detected. Evaluations of verifier-in-the-loop repair should score contract satisfaction, task completion and regression separately, with completion scored on the same inputs as satisfaction.

1 Introduction

Counterexample-guided inductive synthesis (CEGIS) is a long-established pattern in program synthesis [Solar-Lezama et al., 2006]. A synthesizer proposes a candidate, a verifier tries to refute it, and each refutation constrains the next proposal. Recent work substitutes a large language model for the synthesizer, and tests or execution feedback for the verifier [Madaan et al., 2023, Shinn et al., 2023, Chen et al., 2024, Orvalho et al., 2025]. In safety-critical control the natural verifier is not a test suite but a falsifier: a search over scenario parameters for an input on which the policy violates a temporal-logic specification.

That substitution raises a design question usually settled by intuition. A falsifier can return a great deal of structure. It can report which predicate was violated, by what signed margin, at which timestep, under which minimal parameter assignment, at which boundary of the failing region, and which part of the program was responsible. None of it is free: delta debugging and boundary search

NeurIPS 2026 Workshop on AI for Verifiable Coding.

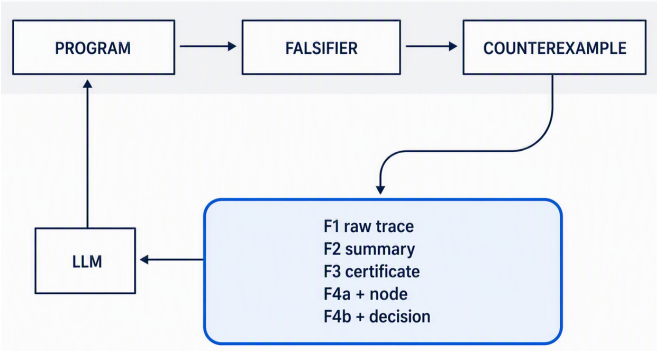


Figure 1: The ablation. Falsifier, frozen program, counterexample corpus, token budget and evaluation suite are identical across all five conditions (grey band). Only the rendering of the failure into text differs (highlighted). The corpus is banked once and read by every condition, so minimization and boundary-search cost are amortized rather than charged to the structured conditions.

cost simulations, and every field costs tokens. The intuition is that more structure repairs better. We set out to test it (Figure 1).

What we found is that the feedback conditions differ sharply in what repair is attempted, that, excluding refusals, we could not detect any of them improving repair success, and that pursuing the second question turned up a more useful one. Our most transferable finding concerns how repairs come to satisfy a specification that asks only for safety.

Contributions.

1. Contract satisfaction on a failing scenario is trivially achievable by refusing to act, and measuring task completion does not guard against this when completion is scored on a different input distribution. We separate three criteria (contracts satisfied, manoeuvre completed, regression suite intact) and show how far apart they are (§4.2).
2. The feedback conditions differ sharply in the kind of repair attempted, a 76.7-point difference in structural-edit rate between the raw-trace and natural-language conditions. The raw trace also differs in length and information content, so we do not attribute this to formal structure (§4.1).
3. A constructive test of what the language permits. Hand-written repairs isolate three mechanisms, two in the baseline and one in a language predicate, that defeat natural small edits, and the single clean repair found by the model shows that the language can express a program that resolves and completes all twenty counterexamples (§4.3).
4. No detected improvement in repair success once refusals are excluded, with intervals that treat counterexamples as the independent units and a detectable-effect estimate under stated assumptions (§4.4), and a structural precondition, $H(\text{node} \mid \text{contract}) > 0$, for when blame localization can carry information at all (§4.6).

2 Related work

LLMs that write control programs. Code-as-policies approaches have language models emit executable policies [Liang et al., 2023], with prompt construction following the import-block convention of ProgPrompt [Singh et al., 2023]. These synthesize but do not falsify.

Adversarial scenario generation. Falsification and adversarial scenario search for driving policies are well established [Wang et al., 2021], with declarative scenario specification following Scenic [Fremont et al., 2019]. These falsify, but where they repair at all they do so by retraining weights or augmenting data.

Table 1: The five contracts, in signal temporal logic with space robustness [Donzé and Maler, 2010]. $\rho = \min_i \rho_i$, so the argmin gives both the weakest contract and when it is weakest. ρ_4 uses the safe longitudinal distance of Responsibility-Sensitive Safety (RSS) [Shalev-Shwartz et al., 2017]; ρ_2 is that work's rule 3 and the avoidability filter its rule 5.

	Contract	**Robustness**	**Threshold**
ρ_1	no collision	metres of bounding-box clearance	≥ 0
ρ_2	yield to oncoming	seconds of margin above τ_{yield}	3.0 s
ρ_3	do not enter on red	seconds of green remaining at entry	≥ 0
ρ_4	RSS distance to lead	metres above the safe distance	RSS Lemma 2
ρ_5	kinematic comfort	m/s^2 of headroom	3.0 m/s^2

Counterexample-guided LLM repair. Closest to this work, Orvalho et al. [2025] run a CEGIS loop with an LLM synthesizer over introductory programming assignments, using a test suite as the oracle and MaxSAT-based fault localization to produce a bug-free program sketch. They find that counterexample guidance substantially improves repair across six models and yields smaller fixes. They ask whether counterexample feedback helps at all; we hold the counterexample fixed and vary its rendering, so our failure to detect an improvement is not in tension with their result. Their oracle is a test suite over discrete inputs, where ours is a robustness falsifier over a continuous scenario space, and their programs are scored only on correctness, whereas ours must satisfy a safety specification *and* complete a task, which is the tension §4.2 turns on.

LLM self-repair. Self-Refine [Madaan et al., 2023], Reflexion [Shinn et al., 2023] and Self-Debugging [Chen et al., 2024] iterate on free-form or test-derived feedback. Two of their results shaped this design. Removing Reflexion's natural-language self-reflection returns pass@1 exactly to the base model's 0.60, against 0.68 for full Reflexion, so every structured condition here also carries a verbalized narrative. Self-Debugging reports that adding an execution trace to unit-test feedback *lowers* accuracy on TransCoder (87.9% against 91.6%), so we did not assume more structure would help.

3 Method

3.1 Task, simulator, and contracts

The task is an unprotected left turn, a permissive circular green rather than a green arrow, at a fixed signalized four-way junction in CARLA [Dosovitskiy et al., 2017] Town05, recorded by exact junction identifier. The simulator runs synchronously at 20 Hz with fixed physics substepping. Sensing is ground-truth state and weather is fixed.

Reproducibility, stated precisely. The ego trajectory reproduces bit-identically under identical scenario parameters, within a process and across restarts, and four of the five contracts reproduce exactly in controlled repeats. The exception is ρ_4, which varies by up to 0.69 m because the lead vehicle's braking trigger reads its position back from the simulator. ρ_4 was the binding contract on the counterexample in 0 of 300 attempts.

Conjunction as minimum (Table 1) gives blame localization for free, and masks every contract except the weakest. We therefore report per-contract robustness rather than ρ alone; §4.5 gives a result invisible under the minimum.

3.2 Program representation and baseline

The policy is a behaviour tree in a small domain-specific language (DSL), parsed to an AST we own and compiled to `py_trees`. Owning the AST is what enables tree edit distance, node-level blame, and validation before execution. Composites compile without memory, so every predicate is re-evaluated from the root on each tick.

The baseline is LLM-synthesized from one fixed prompt, sampled 18 times; two responses were truncated at the $16{,}000$-token output cap then in use, and 14 of the other 16 were valid programs.

Table 2: The manipulated variable. All five share one template with a single feedback placeholder; the 10,874 bytes of prefix and 102 bytes of suffix are verified byte-identical across all conditions and certificates.

Condition	Feedback block	Prompt tokens (med.)
F1	raw trajectory excerpt around the violation	16,382
F2	natural-language narrative only	4,584
F3	contract, ρ, per-contract ρ_i, minimal params, boundary, narrative	4,780
F4a	F3 plus `active_node`	4,871
F4b	F3 plus `authorising_node`	4,867

One was *selected* from these against two gates measured on a shared 100-scenario calibration set: a violation rate in $[0.20, 0.40]$, and task completion above 60%, the latter because a program can reach a low violation rate simply by never entering the junction. The frozen baseline violates on 20 of the 100 calibration scenarios, enters the junction in all of them, and completes the turn in 90. Violation rates for all 14 candidates are in the released data.

3.3 Falsification, certificates, and the five conditions

The falsifier samples a six-parameter scenario space uniformly, then refines locally around low-robustness points, ranking on per-contract robustness rather than ρ, because the minimum can cancel a parameter that moves ρ_1 and ρ_2 in opposite directions. Plausibility constraints are enforced at sample time following Scenic [Fremont et al., 2019]. Each admitted counterexample is minimal (delta-debugged [Zeller and Hildebrandt, 2002]), generalized (each surviving parameter binary-searched to where ρ crosses zero; seven of these boundaries are withheld, §5), actionable (a violation-free response existed), and verbalized. We banked $K{=}20$ counterexamples at a mean cost of 26.6 simulations each.

F1 is not a subset of the other conditions (Table 2) in either direction. The raw trace carries the behaviour-tree node columns, so it contains the blame signal F2 and F3 lack, unlabelled and unaggregated but present. It does not name the violated contract, and carries no robustness value, scenario parameter or boundary. An unwindowed F1 prompt measures 202,812 tokens at the median, $10.1\times$ the per-attempt budget of 20,000, so it must be windowed. The largest window we tested that fits every certificate (± 10 rows) cannot reach the transition F4b names, which lies 28 to 47 rows back in all twenty cases. The labels F1 to F4b order the conditions by how much the failure is processed, not by how much information they carry (Table 5 in Appendix A).

Scope, as planned and as run. Our internal study plan pre-specified three contracts and four scenario parameters; we ran five and six, a change made before the baseline was frozen and explained in Appendix B. The pre-specified blame field is `active_node`. The field `authorising_node` was defined after observing that the first was constant; it is exploratory, and no claim in this paper rests on F4b outperforming F3.

Generation settings. All 300 repair calls use `claude-opus-5` at effort `high`, streaming, with `max_tokens` $= 48{,}000$ and prompt caching on the invariant system block. The model exposes no temperature, `top_p` or `top_k` parameter; `effort` is the only sampling control and is held constant. The served model identifier is logged per call alongside the requested one, and both were `claude-opus-5` on all 300 calls. The replicate index is not a sampling seed: replicates are independent samples, not reproducible ones. We measured 3.00 distinct programs per (condition, certificate) cell out of 3. The prompt budget $B = 20{,}000$ caps the input and is what forces F1 to be windowed; `max_tokens` caps the output, and no response reached it.

4 Results

Five conditions, twenty certificates and three replicates give 300 repair attempts. Each patched program is evaluated on 61 scenarios: the counterexample, a 30-scenario regression suite the baseline passes, and a 30-scenario held-out suite drawn from a reserved interior region the falsifier never sampled. Outcomes fall in three buckets never pooled: *truncated* (generation hit the output cap, a

harness failure), *malformed* (validator rejection), and *ok*. Across all 300 attempts we observed no malformed and no truncated outputs, so every failure is a valid program that did not fix the problem.

Uncertainty. All five conditions are run on the same twenty counterexamples, three times each, so attempts are not independent and we treat the counterexample as the unit of analysis. Every interval in the main text is a 95% percentile interval from a paired cluster bootstrap: each of $10{,}000$ resamples draws twenty counterexamples with replacement, keeps all fifteen attempts on each, and takes differences between conditions within that resample, so two conditions are always compared on the same cases. We apply no correction for multiple comparisons; under a Bonferroni correction over every difference we compute, only the structural-edit differences of F1 from each other condition still exclude zero (Appendix D).

4.1 The feedback conditions differ in the kind of repair attempted

Every patch is classified automatically, with no manual labelling, by comparing the *skeleton* of the patched program, the ordered tree of node names with all parameters removed, against the baseline's. A patch that changes the skeleton is a *skeleton edit*; one that keeps it and changes only numeric parameters, thresholds or set-points, is a *hole fill*. The categories and the classifier were fixed before the campaign (Appendix C), but comparing their rates across conditions was not a recorded prediction, so we report this contrast as an observed effect rather than a confirmed one. It is the largest and most robust condition-dependent effect we measure.

Skeleton edits numbered 59, 13, 15, 15 and 28 of 60 under F1 to F4b (Figure 4, Appendix C): the model restructured the program in 98.3% of attempts under the raw trace and in 21.7% under the natural-language summary, a difference of 76.7 points with 95% interval $[+61.7, +90.0]$. F4b sits between the others at 46.7%; its interval against F3, $[-3.3, +45.0]$, includes zero.

The robust claim is narrower than "more structure gives more control", and narrower than "formal structure changes the edit". Edit rates are not monotone in structure: the least processed condition produces by far the most structural edits, while F2, F3 and F4a, which add structure step by step, differ by at most two. More importantly, F1 differs from every other condition in three ways at once. It is unprocessed, it is roughly 3.5 times as long ($16{,}382$ against $4{,}584$ to $4{,}871$ median prompt tokens, Table 2), and it carries different information (§3.3). What this design can establish is that the raw-trace condition differs from every processed condition, by 51.7 to 76.7 points with every interval excluding zero (Appendix D). It cannot say which of the three differences is responsible. A token-matched control would not settle this either: equalizing length leaves the information difference in place, and truncating the trace to match would change what it contains.

4.2 Contract satisfaction is trivially achievable by refusing to act

The success criterion pre-specified in our internal study plan is $\rho \geq 0$ on the failing scenario after the patch. We report it as one of three criteria, each scored independently on every attempt (Table 3): whether the contracts are satisfied on the counterexample (S; we then say the patch *resolves* it), whether the manoeuvre is completed on it (C), and whether the regression suite is left intact (R). A program that never enters the conflict zone satisfies S for free: ρ_2 and ρ_3 become vacuous, since one cannot fail to yield without entering, and are excluded from the argmin; ρ_1 is satisfied because no actor is ever nearby; and ρ falls through to ρ_5, which every such program passed.

This is the synthesis-side analogue of *vacuous satisfaction* in model checking, where a specification passes for reasons unrelated to the behaviour it was written to constrain, and where detecting such passes is a standard obligation rather than an afterthought [Beer et al., 1997, Kupferman and Vardi, 2003]. Vacuity detection asks whether a formula held trivially; the analogous question for repair is whether a patch satisfied the contracts by making them inapplicable, and on this corpus 13 of the 18 that satisfied them did.

Applied in turn, the three criteria select 18, then 3, then 1 of 300 attempts (Figure 2). Thirteen of the 18 that satisfied the contracts, 72%, never entered the conflict zone; two more entered but had not completed the turn when the episode ended; three completed it, and of those three only one also left the regression suite intact. Across 300 attempts there is exactly one clean repair.

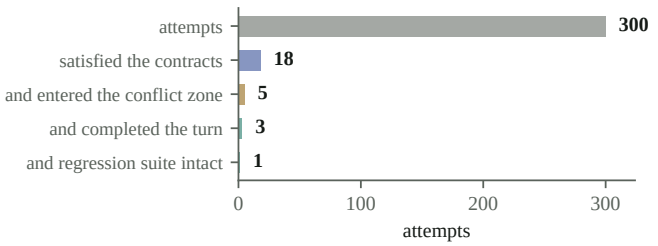


Figure 2: From 300 attempts to one clean repair: 13 of the 18 that satisfied the contracts never entered the conflict zone. The unpatched baseline entered it on all 20 counterexamples, so the refusals are introduced by the patches rather than inherited.

Table 3: Three success criteria, scored independently on every attempt, as counts out of 60 per condition. **S**: every contract satisfied on the counterexample, the pre-specified criterion, split by whether the vehicle entered the junction's conflict zone, a 12 m disc around its centre. **C**: the turn completed on the counterexample, reaching the junction exit before the episode ends. **R**: none of the 30 regression scenarios, all passed by the unpatched baseline, now violates a contract.

	S: contracts satisfied			**C**	**R**	**Jointly**	
Cond.	all	refused	entered	completed	regr. intact	S, C	S, C, R
F1	13	10	3	7	18	1	1
F2	1	1	0	6	23	0	0
F3	3	1	2	7	16	2	0
F4a	0	0	0	7	6	0	0
F4b	1	1	0	6	13	0	0
All	18	13	5	33	76	3	1

No single criterion identifies a repair, which is why we report all three. R is itself a contract check on other scenarios, so a program that does not drive passes it too: one patched program completed none of the 60 suite scenarios and caused no regression failure. C alone is no better a measure of repair, since 30 of its 33 successes are on the two counterexamples the unpatched baseline already completed. Only the conjunction identifies a repair.

The obvious defence is to measure task completion as well, and in the form we pre-specified it does not work. Our pre-specified completion metrics were computed over the 60-scenario evaluation suite, while success was computed on the single counterexample. Suite completion does catch a program that refuses everywhere, like the one above, but the other twelve refusals completed 93 to 95% of suite scenarios, against 95% for the unpatched baseline. No pre-specified metric measured completion on the scenario the success metric reads from; C in Table 3 does. Any evaluation that scores specification satisfaction on one input distribution and task completion on another inherits this defect.

A proposed remedy, untested. One candidate fix is in the specification rather than the feedback: require completion on the failing scenario, or run a vacuity check in the model-checking sense and return its verdict to the model. We have not tested either. Whether they reduce resolution by refusal, or move it to a different degenerate answer, is future work.

4.3 What the language permits, and why repairs may resolve by refusal

Because every attempt produced a valid program, the failures leave open whether the language could express a correct repair at all. We tested this constructively, writing candidate repairs by hand in the same vocabulary and running them on all twenty counterexamples (Table 4).

Three mechanisms defeat the natural small edits, two in the baseline and one in the language's yield predicate, each measured rather than inferred. First, the baseline's committing branch is guarded only by a stop-line distance test satisfied once the ego is *past* the line, and it outranks the gap-

Table 4: Constructive test of expressiveness. Each row is a valid program, or a family of them, in the same DSL run on all twenty counterexamples. The final row is the single attempt of 300 that resolved its counterexample, completed the manoeuvre and caused no regression, re-evaluated here across the whole corpus; six repeats on its own counterexample reproduced ρ and ρ_4 with zero spread.

Repair	Resolved	Completed	Why
frozen baseline	0/20	2/20	
raise gap thresholds (e.g. $5.5 \rightarrow 9.0$)	0/20	3/20	branch never reached
gap guard on committing branch, up to 15.0	0/20	2/20	guard passes at every value
geometric guard on committing branch	0/20	2/20	blocks, ego creeps in anyway
geometric guard + true stop	20/20	0/20	never enters the junction
the model's clean repair, from F1	**20/20**	**20/20**	

guarded branch, so the gap thresholds are unreachable and raising them changes nothing. Second, the yield predicate is structurally blind to a stationary obstacle: it reports time-to-arrival and, to keep the arithmetic well defined, excludes any vehicle slower than 0.5 m/s, so in one failing scenario a vehicle 9.4 m away at 0.04 m/s is discarded, the predicate returns its 30 s cap, and a guard demanding 15 s of gap passes while that car occupies the junction. Third, the geometric predicate does see it and does block the turn, but the fallback decelerates to 1.0 m/s rather than to rest, so the ego creeps across the boundary two seconds later and an unguarded branch commits.

Adding a true stop to the geometric guard resolves all twenty counterexamples and completes none of them. That is the degenerate repair.

A safe and completing repair is nonetheless expressible here. One attempt in 300, under the raw-trace condition F1, produced a program that resolved its counterexample, completed the manoeuvre and caused no regression failure. Re-evaluated across all twenty counterexamples, including the nineteen it was never shown, **it resolves and completes every one**. Its patch adds a 12 m path check to the committing branch, lengthens the two path checks at the stop line to 18 m and brakes the fallback harder (2.5 against 2.0 m/s^2) while leaving it able to creep. Our hand-written path checks on the same branch (16 to 32 m, fallback unchanged) fixed none of the twenty, so which of these differences matters is not established. Its own comment identifies the second mechanism: a stalled vehicle reports an effectively infinite time to arrival, so the gap test cannot see it.

What this establishes is narrow. The failures we observe are not explained by the vocabulary being unable to state such a repair, although its yield predicate defeats the most natural one. It does not establish that search is the bottleneck, let alone the only one, and a single success in 300 attempts says little about how hard that search is. Two explanations for why most resolutions were refusals remain open, and our data cannot separate them. The first is the starting point: the mechanisms above defeat the natural small edits to this program in this language, leaving a guard with a true stop as the only hand-written edit that resolved anything; with a different branch ordering this might not hold. The second is the criterion: a specification that rewards only safety gives refusal full credit, so nothing in the feedback penalizes it. A second baseline would test the program-specific part of the first, and a specification that rejects vacuous satisfaction would test the second (§4.2). We ran neither.

4.4 Repair success: no detected improvement

After excluding refusals (the 'entered' column of Table 3) we did not detect a reliable improvement in repair success from any feedback condition (Figure 3). Every paired comparison against F3 has an interval containing zero; for F2, F4a and F4b, which have no such successes, zero is the upper endpoint. An interval containing zero is not evidence that the effect is absent, only that this design did not detect one. The raw-trace condition clears zero on the pre-specified rate, but 10 of its 13 resolved attempts are refusals, and excluding those the difference is $+1.7$ with an interval of $[-5.0, +8.3]$.

We make no significance claim about improvement in repair success, and we do not claim a null. With $K{=}20$ counterexamples, three replicates and between zero and thirteen successes per condi-

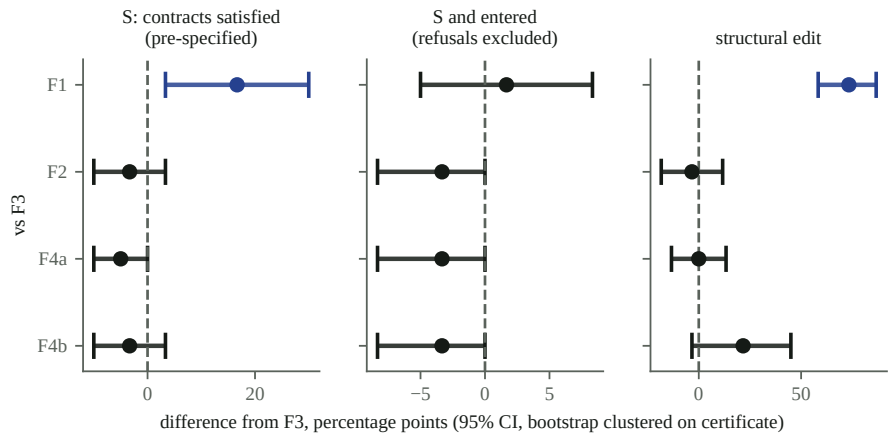


Figure 3: Paired differences against F3 with 95% intervals from a bootstrap clustered on certificate, which respects both the shared counterexample corpus and the three repeated attempts per cell. Only the raw-trace condition separates from zero (an interval ending exactly at zero, where a condition has no successes, does not), and on repair success that separation is an artefact of refusal: excluding refusals, its interval includes zero like every other.

tion, the design supports neither. Appendix E estimates by simulation how large a true improvement this design could detect. Under the assumptions stated there, an improvement over F3's 3.3% is detected with probability 0.8 only once it reaches 13 to 20 percentage points, and one of 5 points at most a quarter of the time. These figures describe the design under those assumptions; they are not a measured detection limit.

4.5 Repairs were net destructive, and the minimum hides part of it

Across all 300 attempts, 224 (74.7%) caused at least one previously passing regression scenario to fail, against 18 that satisfied the contracts. The modal outcome of a repair attempt is that it did not fix the counterexample and broke one thing that worked. This holds in every condition. Besides the F1 contrasts, one other interval excludes zero without adjustment: F4a broke at least one regression scenario more often than F3, by $+16.7$ points $[+3.3, +30.0]$. Among 22 unadjusted intervals about one would exclude zero by chance, and this one does not survive the Bonferroni correction; since F4a carries no information beyond F3, a real effect would be one of redundant text, not of blame. No condition returned regression results to the model, so an iterated loop that does might behave differently, but as measured here these patches should not be applied without regression checking.

Per-contract reporting reveals a second effect invisible under the minimum: mean ρ_5 on the counterexample falls in every condition, from 1.000 to between 0.59 and 0.93, so patches on average command harsher acceleration or braking, and whenever ρ_1 is the argmin this is masked completely.

4.6 When blame localization can carry information

Assigning blame to the leaf executing at the argmin is vacuous for a program like ours, whose highest-priority branch is a region test followed by a single action, whenever the violated contracts are evaluated over that same region, because only that action can run there. On our baseline $H(\texttt{active_node} \mid \text{contract}) = 0.000$ bits against $H(\text{contract}) = 1.234$ bits, and we verified by construction that five contracts each blaming one node still give zero, so this is a property of program structure, not of contract count. We therefore propose a precondition, cheap to check before running such an experiment: $H(\text{node} \mid \text{contract}) > 0$. Where it fails, the blame field re-encodes the contract label. It may still influence the model, as redundant text can, but it carries no information beyond the label. Defining blame instead as the last distinct leaf active before the failing leaf began running, the decision that authorized the action rather than the action executing when the consequence landed, raises this to 0.500 bits; that definition is exploratory (§3.3).

5 Limitations

Two limitations deserve more than a clause. First, **every result is conditioned on a single frozen baseline program**, one model (`claude-opus-5`) and one repair step. The structural argument of §4.6 should carry to other programs of that shape, but no effect size here is claimed to transfer to another program. A second baseline would test that directly and is the most valuable single extension we can identify; it would also tell us whether the refusal result of §4.2 is a property of this program's structure or of the task and its success criterion. Second, **the expressiveness audit is not exhaustive**. Our hand-written repairs were a family of natural single- and double-edit candidates, not a search of the program space, and the model found a repair that family missed, so Table 4 shows which natural edits fail, not which edits the DSL rules out.

The remaining limitations are ordinary. $K{=}20$ with three replicates supports no significance claim about improvement in repair success, and under the assumptions of Appendix E only large improvements in repair success were reliably detectable. We measure one repair step per counterexample, with no regression feedback, so that every condition patches the same frozen program; how effects compound over rounds is untested. F1 differs from the other conditions in length and information content as well as in structure, so the edit-type contrast cannot be attributed to any one of the three, and we ran no length-matched control (§4.1). The specification change proposed in §4.2 is untested. Seven of twenty certificates report no boundary, because their sole boundary parameter was measured to be anti-correlated with the quantity it names and was suppressed; those degrade to point counterexamples. Corpus construction is amortized across all five conditions, so the cost asymmetry a deployed closed loop would face is absent here. The simulator side reproduces, apart from ρ_4, but the model does not: it exposes no seed or temperature, and its identifier names no snapshot. Oncoming vehicles follow a scripted policy, holding their speed and, depending on an aggressiveness parameter, braking when the ego nears the conflict zone, which is simpler than naturalistic traffic; sensing is ground truth.

Broader impact. This is a simulation study, and nothing in it is evidence of real-world driving safety: contract satisfaction in CARLA, under ground-truth sensing and simplified traffic, says little about a deployed vehicle. The refusal result is itself a safety concern. A repair objective that rewards only the absence of violations can yield systems that pass every contract-based check while failing their task, and in real traffic a vehicle that never proceeds creates hazards these contracts do not measure. The model's patches also broke previously passing scenarios in three quarters of attempts, so LLM-proposed repairs to safety-critical code need independent verification before use.

6 Conclusion

The feedback conditions we compared differ sharply in what repair is attempted, but because the raw trace differs in length and information content as well as in structure, this is a difference between conditions, not a measured effect of formal structure. Excluding refusals, we detected no condition improving repair success, which at this sample size is not evidence that no such effect exists.

The more useful finding came from asking why so many resolved attempts never entered the conflict zone. Contract satisfaction on a failing scenario is trivially achievable by inaction, and neither completion scored on other scenarios nor a contract-based regression suite reliably catches it. Applied in turn, contracts satisfied, manoeuvre completed and regression suite intact select 18, then 3, then 1 of 300 attempts, and a program meeting all three exists in this language for this corpus.

We conjecture that where a repair task scores only specification satisfaction and the program representation admits inaction, the repairs that satisfy it will tend to be refusals, though our data cannot separate this from the structure of the one baseline we repaired. A second baseline, and a specification that rejects vacuous satisfaction, would test it.

Acknowledgments and Disclosure of Funding

This work received no external funding. N.M. designed the study, built the system, ran all experiments and analyses, and wrote the paper; S.P. advised. Figure 1 was drawn with an AI image-generation tool from a schematic the authors specified; it carries no data. Code, the frozen program,

the twenty counterexample certificates, all prompts and raw model responses, and the scripts and saved outputs behind every table and figure are released at `https://github.com/mistryneel/counterexample-feedback` (release `v1.1-camera-ready`); the version that was reviewed is release `v1.0-submission`.

References

Ilan Beer, Shoham Ben-David, Cindy Eisner, and Yoav Rodeh. Efficient detection of vacuity in ACTL formulas. In *Computer Aided Verification (CAV)*, volume 1254 of *Lecture Notes in Computer Science*, pages 279–290. Springer, 1997.

Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. In *International Conference on Learning Representations (ICLR)*, 2024.

Alexandre Donzé and Oded Maler. Robust satisfaction of temporal logic over real-valued signals. In *Formal Modeling and Analysis of Timed Systems (FORMATS)*, volume 6246 of *Lecture Notes in Computer Science*, pages 92–106. Springer, 2010.

Alexey Dosovitskiy, German Ros, Felipe Codevilla, Antonio López, and Vladlen Koltun. CARLA: An open urban driving simulator. In *Proceedings of the 1st Annual Conference on Robot Learning (CoRL)*, volume 78 of *Proceedings of Machine Learning Research*, pages 1–16, 2017.

Daniel J. Fremont, Tommaso Dreossi, Shromona Ghosh, Xiangyu Yue, Alberto L. Sangiovanni-Vincentelli, and Sanjit A. Seshia. Scenic: A language for scenario specification and scene generation. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 63–78, 2019.

Orna Kupferman and Moshe Y. Vardi. Vacuity detection in temporal model checking. *International Journal on Software Tools for Technology Transfer (STTT)*, 4(2):224–233, 2003.

Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 9493–9500, 2023.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-Refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 46534–46594, 2023.

Pedro Orvalho, Mikoláš Janota, and Vasco Manquinho. Counterexample guided program repair using zero-shot learning and MaxSAT-based fault localization. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, 2025.

Shai Shalev-Shwartz, Shaked Shammah, and Amnon Shashua. On a formal model of safe and scalable self-driving cars. *arXiv preprint arXiv:1708.06374*, 2017.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox, Jesse Thomason, and Animesh Garg. ProgPrompt: Generating situated robot task plans using large language models. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 11523–11530, 2023.

Armando Solar-Lezama, Liviu Tancau, Rastislav Bodík, Sanjit Seshia, and Vijay Saraswat. Combinatorial sketching for finite programs. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 404–415, 2006.

Jingkang Wang, Ava Pun, James Tu, Sivabalan Manivasagam, Abbas Sadat, Sergio Casas, Mengye Ren, and Raquel Urtasun. AdvSim: Generating safety-critical scenarios for self-driving vehicles. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 9909–9918, 2021.

Andreas Zeller and Ralf Hildebrandt. Simplifying and isolating failure-inducing input. *IEEE Transactions on Software Engineering*, 28(2):183–200, 2002.

A What each condition carries

Table 5: Information content of each feedback block. F3 is F2's narrative plus structured fields, and F4a and F4b each add one blame field to F3, so F2 is contained in F3 and F3 in each of F4a and F4b. F1 is not nested with any of them. It carries the per-row behaviour-tree columns, from which the leaf executing at the violation can be read, but it does not name the violated contract, does not mark the failing row, and carries no robustness value, parameter or boundary. The transition that F4b names lies 28 to 47 rows before the violation, outside F1's 21-row window, so F1 cannot show it. *Median prompt length minus the 4,513 tokens of scaffolding shared by every prompt, so approximate.

Information	**F1**	**F2**	**F3**	**F4a**	**F4b**
feedback block, median tokens*	11,869	71	267	358	354
raw trajectory, 21 rows × 45 columns around the violation	✓				
behaviour-tree leaf ticking at each row (unlabelled columns)	✓				
every actor's role, position and velocity, per row	✓				
what was violated, by how much, and when		prose	✓	✓	✓
gap the ego actually faced		prose	prose	prose	prose
named contract, ρ and every ρ_i			✓	✓	✓
minimal parameters and irrelevant parameters			✓	✓	✓
boundary of the failing region (13 of 20 certificates)			✓	✓	✓
natural-language narrative		✓	✓	✓	✓
leaf executing at the violation, named				✓	
leaf that authorized the failing action, named					✓

B Deviations from the internal study plan

Our internal study plan, released with the code at `https://github.com/mistryneel/counterexample-feedback/tree/v1.1-camera-ready/docs/study_plan`, pre-specified three contracts and four scenario parameters. We ran five and six. The change was made before the baseline was frozen and for a measured reason: on the reference program under three contracts, the violated contract determined the blamed leaf exactly, so blame localization carried no information beyond the contract label and the F3/F4 contrast would have been vacuous by construction. We re-enabled the red-light and lead-distance contracts to widen the range of failure modes. Of the four planned parameters, the pedestrian-phase offset was dropped because no pedestrian was ever spawned, so it had no effect; the signal offset and lead-vehicle gap were added because they make the new contracts bind; and an aggressiveness parameter for oncoming vehicles was added because without it only one parameter carried signal and every boundary collapsed onto a single axis. We report this because it is a deviation from the plan, and because it did not have the intended effect: adding contracts does not by itself break the collapse, since each contract may still map to a single leaf. What addressed it was the alternative blame definition of §4.6, which is exploratory and labelled as such throughout. The pre-specified field is `active_node`; `authorising_node` was defined after observing that the first was constant, and no claim in this paper rests on F4b outperforming F3.

The pre-specified success criterion was $\rho \geq 0$ on the counterexample scenario. We still report it, but after measuring that most of its successes were refusals (§4.2) we lead with the three criteria separately rather than replacing it, so that the change is visible rather than silent.

C Edit taxonomy

Patch type is a deterministic function of the baseline and patched sources (`classify_patch` in `src/ted.py`); no attempt was labelled by hand. The skeleton of a parsed program, comments discarded, is the tree of node names with every parameter removed and child order kept. If the

skeletons differ, the patch is a skeleton edit, which covers inserting, deleting or reordering a node and replacing one node type with another. If they agree, it is a hole fill when any parameter value differs after rounding to six decimals, and no change otherwise; a program that fails to parse is unparseable. Only outputs that pass the validator are classified.

The two categories, and the hypothesis that skeleton edits carry more regression risk, were in the design specification before any repair attempt. The classifier was unit tested and committed before the campaign, which ran from that commit; apart from comments, it has not changed since. Because the label is computed, inter-rater reliability does not arise; re-running the classifier on the logged programs reproduces all 300 labels. Comparing edit-type rates across conditions was not among the expectations we recorded in advance, which is why §4.1 reports it as an observed effect.

There were 170 hole fills, 130 skeleton edits, and no no-change or unparseable outputs; Figure 4 shows the split by condition. Every skeleton edit increased the number of nodes, so no label rests on a rename or a reordering alone. Of the hole fills, 140 changed at least one action parameter, such as a target speed, and 30 changed only predicate thresholds. Hole fills are not smaller edits: their median tree edit distance is 8 against 5 for skeleton edits, so the taxonomy measures the kind of edit, not its size. Mean tree edit distance per patch was 5.93, 5.77, 6.95, 7.38 and 4.27 for F1 to F4b. Skeleton edits averaged 0.94 regression failures against 0.81 for hole fills; patch type is an outcome confounded with condition, so we report this descriptively and not as a test of the regression-risk hypothesis.

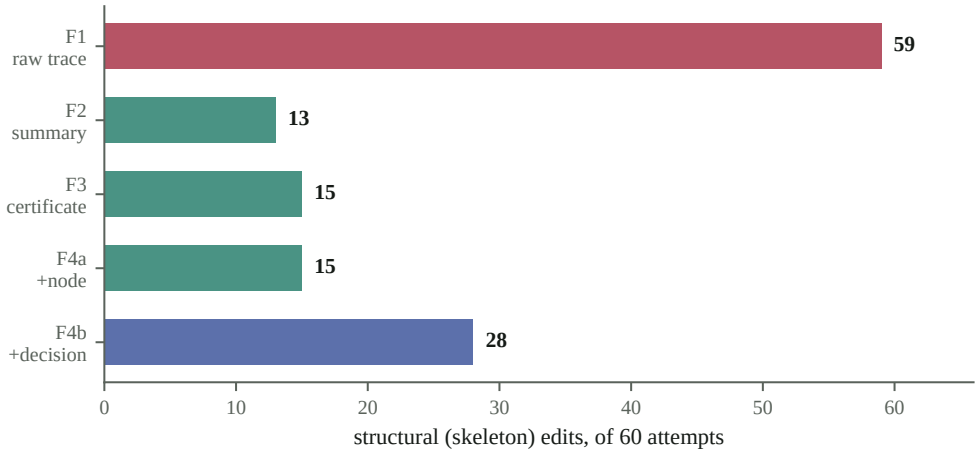


Figure 4: Structural edits by condition, out of 60 attempts each. Under the raw trace the model restructured the program in almost every attempt; under a natural-language summary it mostly changed parameter values. F1 also differs from the other conditions in length and information content (§4.1).

D Paired cluster bootstrap

Unit. The resampling unit is the counterexample certificate. There are $K = 20$, each attempted three times under each of the five conditions, so every certificate carries 15 attempts and the design is balanced. Attempts on the same certificate share its difficulty, and every condition is run on the same certificates, so neither attempts nor conditions are independent; certificates are.

Procedure. Each of $10{,}000$ resamples draws 20 certificates uniformly with replacement and keeps every attempt on each drawn certificate. For each condition we compute the percentage of its attempts in the resample that meet the outcome; the denominator is always 60. A difference between two conditions is taken within the same resample, so both are always evaluated on the same certificates. The 95% interval runs from the 2.5th to the 97.5th percentile of the $10{,}000$ differences (NumPy PCG64, seed 0, linear interpolation). Because the design is balanced, this equals a percentile bootstrap of the mean of the 20 certificate-level differences $100\,(s_{i,a} - s_{i,b})/3$, where $s_{i,a}$

counts attempts on certificate i under condition a that meet the outcome. Figure 3 plots intervals from the same resamples.

Outcomes. A *structural edit* is a skeleton edit (Appendix C). *Success excluding refusals* requires $\rho \geq 0$ on the counterexample and entry into the conflict zone; a refusal stays in the denominator as a failure.

Comparisons. We tabulate 17 paired differences: each condition against F3 on four outcomes, and F1 against F2 on structural edits (Table 6). The intervals in the paper are unadjusted. Under a Bonferroni adjustment over these 17, the structural-edit differences of F1 from F2 and from F3 still exclude zero; the two others that exclude zero unadjusted, F1 against F3 on pre-specified success and F4a against F3 on regressions, do not. F1 also differs from F4a and F4b on structural edits, by $+73.3$ $[+53.3, +90.0]$ and $+51.7$ $[+36.7, +66.7]$ in the same resamples. Counting the five further structural-edit pairs, we computed 22 distinct differences; under a Bonferroni correction over all 22, exactly the four F1 structural-edit contrasts exclude zero. F1 against F2, the contrast quoted in the abstract, is the largest of the ten pairwise structural-edit differences, and was singled out after seeing the data.

Limits. With 20 clusters, percentile intervals can be somewhat too narrow. Where a condition has no successes, its difference from F3 cannot be positive, so an upper endpoint of exactly zero reflects absent events rather than a precise estimate. For the joint criteria of Table 3, with zero to two successes per condition, we report counts and no intervals.

Table 6: The 17 tabulated paired differences, in percentage points, from one set of $10{,}000$ certificate resamples (seed 0). Bonferroni intervals use $\alpha = 0.05/17$. Computed by `scripts/uncertainty.py` and saved in `data/results/uncertainty.json`.

Outcome	**Contrast**	**Diff.**	95% **interval**	**Bonferroni** ($\times 17$)
Success (pre-specified)	F1−F3	$+16.7$	$[+3.3, +30.0]$	$[-3.3, +36.7]$
	F2−F3	-3.3	$[-10.0, +3.3]$	$[-13.3, +5.0]$
	F4a−F3	-5.0	$[-10.0, 0.0]$	$[-15.0, 0.0]$
	F4b−F3	-3.3	$[-10.0, +3.3]$	$[-13.3, +6.7]$
Success, refusals excl.	F1−F3	$+1.7$	$[-5.0, +8.3]$	$[-8.3, +13.3]$
	F2−F3	-3.3	$[-8.3, 0.0]$	$[-11.7, 0.0]$
	F4a−F3	-3.3	$[-8.3, 0.0]$	$[-11.7, 0.0]$
	F4b−F3	-3.3	$[-8.3, 0.0]$	$[-11.7, 0.0]$
Structural edit	F1−F3	$+73.3$	$[+58.3, +86.7]$	$[+48.3, +93.3]$
	F2−F3	-3.3	$[-18.3, +11.7]$	$[-26.7, +20.0]$
	F4a−F3	0.0	$[-13.3, +13.3]$	$[-18.3, +20.0]$
	F4b−F3	$+21.7$	$[-3.3, +45.0]$	$[-15.5, +58.3]$
	F1−F2	$+76.7$	$[+61.7, +90.0]$	$[+53.3, +95.0]$
Broke ≥ 1 regression	F1−F3	-3.3	$[-20.0, +13.3]$	$[-26.7, +23.3]$
	F2−F3	-11.7	$[-28.3, +5.0]$	$[-36.7, +15.0]$
	F4a−F3	$+16.7$	$[+3.3, +30.0]$	$[-1.7, +36.7]$
	F4b−F3	$+5.0$	$[-13.3, +20.0]$	$[-25.0, +26.7]$

E Detectable effect under stated assumptions

We estimated how often the decision rule of §4.4 would detect a true difference d in repair success between a condition and F3. Refusals count as failures, so each rate is out of 60 attempts. The rule is the paired cluster bootstrap of Appendix D, with a detection declared when the 95% interval excludes zero. Because the design is balanced, the bootstrap distribution depends only on the twenty per-certificate differences, so we computed it exactly rather than from $10{,}000$ resamples; the two agreed on $5{,}988$ of $6{,}000$ simulated datasets. A certificate's success probability is Beta distributed with mean p_0 under F3 and $p_0 + d$ under the other condition, with intra-certificate correlation (ICC) of replicate outcomes in $\{0, 0.1, 0.3\}$. Its difficulty is either shared across the two conditions, taking the same quantile of each distribution, or independent, and replicates are independent given the

certificate. Table 7 gives the smallest d detected with probability 0.8. Ignoring clustering, a two-proportion calculation for 60 attempts per arm gives 14.8 points (arcsine approximation) to 17.5 points (Fisher's exact test) at $p_0 = 3.3\%$. At $p_0 = 3.3\%$, a true improvement of 5 points is detected with probability 0.17 to 0.23 across these settings.

One property holds whatever the model. The interval cannot exclude zero unless at least four of the twenty certificates favour the condition declared better, so with F3 successful on at most three certificates under each repair-success outcome (S, S excluding refusals, and S with C), no condition could have been shown to do worse than F3 on repair success. Under the model, a Bonferroni correction over the four comparisons against F3 raises each entry by about 3 to 5.5 points, and over all 17 comparisons by about 6 to 10.5. The entries move with the assumptions, most of all with how strongly difficulty is shared, and are properties of the design rather than a measured detection limit. The data cannot estimate either assumption, but the five successes excluding refusals fell one per cell and on five different certificates, a pattern less likely under strongly shared difficulty than under weak clustering, so the lowest entries are the optimistic ones.

Table 7: Smallest true improvement over F3, in percentage points, detected with probability 0.8 by the paired cluster bootstrap over twenty certificates and three replicates, under the model of Appendix E. $p_0 = 3.3\%$ is F3's observed rate both excluding refusals and for S and C jointly; 5.0% is its rate on the pre-specified criterion S. Each setting used $20{,}000$ simulated experiments (`scripts/detectable_effect.py`); repeating every setting with a second seed moved no entry by more than 0.5 points, and entries are rounded to whole points.

	$p_0 = 3.3\%$		$p_0 = 5.0\%$	
ICC	shared	independent	shared	independent
0	15		17	
0.1	14	17	16	19
0.3	13	20	14	22

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and the contributions in §1 present the edit-type contrast as a difference between feedback conditions, explicitly not attributed to formal structure (§4.1), report that, excluding refusals, no improvement in repair success was detected at this sample size (§4.4), and leave open why the one clean repair was found only once (§4.3). The refusal finding is supported by three separately scored criteria (§4.2), where the specification fix is presented as untested future work, and the scope of one baseline, one model and one repair step is stated in §5.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: §5 is a dedicated Limitations section covering the single frozen baseline, single model and single repair step, the non-exhaustive expressiveness audit, the sample size, the length and information confound in the raw-trace condition, the untested specification change, the certificates without a boundary, the amortized cost of corpus construction, the non-reproducible model samples, and the simplified traffic and ground-truth sensing. Deviations from the internal study plan are reported in Appendix B.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.

- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [N/A]

   Justification: The paper states no theorems or formal proofs. The observation in §4.6 that leaf-level blame carries no information beyond the contract label for a program whose highest-priority branch is a region test followed by a single action, when the contracts are evaluated over that region, is an informal structural argument, measured on the baseline and checked on a constructed five-contract case, and $H(\text{node} \mid \text{contract}) > 0$ is proposed as a precondition to check before an experiment, not proved.

   Guidelines:

   - The answer [N/A] means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: §3 specifies the task, simulator settings and contracts (Table 1), the language and the baseline selection gates, the falsifier and certificate construction, the five conditions with byte-identical shared scaffolding (Table 2), and the model and generation settings, while §4 and Appendices A to D define every outcome, the edit-type classifier and the bootstrap. The exact prompt template, baseline program, certificates and scenario bounds are in the released repository rather than the paper, and because the hosted repair model exposes no seed or temperature (§3.3), the released prompts and raw responses are what make the reported results verifiable.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
   - If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
   - Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via

detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.

- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: Code, the frozen baseline, the 20 counterexample certificates, the prompts and raw model responses for all 300 attempts, and the per-attempt logs are released at `https://github.com/mistryneel/counterexample-feedback`, and the counts in Table 3 and Figure 4 and the intervals in Table 6 recompute offline from those logs without the simulator or API access. Re-running simulations requires CARLA 0.9.16, and new repair samples require API access to the hosted `claude-opus-5` model and will not reproduce sample for sample, since it exposes no seed or temperature (§3.3).

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: No model is trained; §3 gives the simulator settings, the baseline selection gates and calibration set, the falsifier and $K=20$, the five conditions with their prompt lengths, the per-attempt prompt budget and the generation settings, and §4 defines the 61-scenario evaluation (the counterexample, a 30-scenario regression suite and a 30-scenario held-out suite from a region the falsifier never sampled). The bounds of the six scenario parameters, each with a written justification, are in the released configuration rather than the paper.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: Every interval is a 95% percentile interval from a paired cluster bootstrap over the 20 counterexample certificates (10,000 resamples, NumPy PCG64 with seed 0), described in §4 and Appendix D; the intervals are unadjusted for multiple comparisons, and Appendix D reports Bonferroni-adjusted intervals for all 17 differences as a sensitivity analysis. They capture variation over which counterexamples are sampled, with the three independent replicate samples per condition and certificate kept together inside each resampled certificate, assume no normality, and Appendix D notes that with 20 clusters they may be somewhat too narrow and that the joint criteria are reported as counts without intervals.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: All simulation ran on one workstation (AMD Ryzen 9 9900X with 12 cores, 30 GiB RAM, NVIDIA RTX 3090 GPUs, CARLA 0.9.16 in Docker rendering off-screen at about 3 GB of GPU memory per server), and the repair model was called through a hosted API; building the corpus took 422 s for 531 simulations, the 300 repair calls took 36.3 minutes run concurrently (2,153,499 input tokens, 1,339,221 of them read from the prompt cache, and 1,369,282 output tokens), and evaluation took 256.2 minutes on one serial simulator over 18,380 rollouts. These figures are given here and in the released environment manifest rather than in the main text; the campaign's API cost was not logged per call, so we report token counts, and the full project used further compute, not tallied, for baseline synthesis and calibration, determinism checks, sensitivity sweeps, a dry run and the expressiveness probes.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

Answer: [Yes]

Justification: The work is a simulation study with no human subjects, no personal or scraped data and no deployed system. Its main foreseeable risk, that simulated contract satisfaction or LLM-proposed patches could be mistaken for evidence of real-world safety, is addressed in the Broader impact paragraph of §5.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: The Broader impact paragraph of §5 states that contract satisfaction in simulation is not evidence of real-world driving safety, that repair objectives rewarding only the absence of violations can yield systems that pass every contract-based check while failing their task, and that LLM-proposed patches broke previously passing scenarios in three quarters of attempts and need independent verification. The positive side, that scoring contract satisfaction, task completion and regression separately exposes such vacuous repairs, is argued in §4.2.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.

- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: We release no pretrained model, image generator or scraped dataset. The released assets are behaviour-tree programs for a simulated vehicle, counterexample certificates, prompts, model responses and simulation logs, none of which poses a high risk for misuse, and §5 states that the programs are not evidence of real-world driving safety.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: We cite CARLA [Dosovitskiy et al., 2017] (§3) and used version 0.9.16 (`https://github.com/carla-simulator/carla`), whose code is under the MIT License and whose assets, including the Town05 map, are under CC-BY according to its README, with Unreal Engine under its own terms; we redistribute no CARLA binaries or assets, only trajectories logged from the simulator. We also use py_trees 2.5.0 and rtamt 0.3.5 (both BSD 3-clause; rtamt only cross-checks our robustness values), NumPy 2.5.2 (BSD-3-Clause, with bundled parts under 0BSD, MIT, Zlib and CC0-1.0), SciPy 1.18.1, pandas 3.0.5 and networkx 3.6.1 (BSD-3-Clause), pyarrow 25.0.1 (Apache-2.0), PyYAML 6.0.3 (MIT), matplotlib 3.11.1 (the PSF-based matplotlib licence) and the Anthropic Python SDK 1.2.0 (MIT), and we accessed `claude-opus-5` through its provider's API.

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.

- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: The new assets are the experiment code with its behaviour-tree language toolchain, the frozen baseline program, the 20 counterexample certificates, the prompts and raw model responses for all 300 attempts, and the per-attempt evaluation logs, released at `https://github.com/mistryneel/counterexample-feedback` with documentation of the data schemas, the scenario bounds with a written justification for each, and the pinned software environment. Code is released under the MIT License and data under CC BY 4.0. No asset contains data about people.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: The paper involves no crowdsourcing and no human subjects; every experiment is a simulation or a call to a language model.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: There were no study participants, so participant risk and IRB approval do not apply.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does not impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: The LLM is the core of the method: `claude-opus-5` synthesized the candidate programs from which the frozen baseline was selected (§3) and produced all 300 repair attempts. §3.3 states the model, its generation settings, the prompt budget, and that replicates are independent rather than reproducible samples. Separately, Figure 1, a schematic carrying no data, was drawn with an AI image-generation tool, as stated in the acknowledgements.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.